R4.11 Conception et développement d'applications mobiles



Jean-François Remm

2025–2026

IUT Nantes département Informatique

Objectif

Extrait du PN

- Problématiques de la mobilité (par ex. : autonomie, robustesse)
- Programmation pour un système mobile
- Connectivité, utilisation de "Services Web" (*Web services*)

Organisation du module

Volume horaire : $\simeq$20H

- 4H CM + 12H TD + 4H TD SAE

Modalité d'évaluation :

- note de miniprojet $\frac{1}{4}$
- note de test $\frac{3}{4}$

Équipe pédagogique

- CM : Jean-François Remm
- TD : Haj Ali Ibtissem, Arnaud Lanoix et Jean-François Remm

Développement Mobile

- application mobile = application développée pour :
  - smartphone,
  - tablette,
  - montres connectées (*Wear*),
  - véhicules,
  - ...
- écosystèmes complets (OS, “market”, SDK, ...) :
  1. android
  2. ios
  3. ...
- développement mobile :
  1. application web : html5 + css + js
  2. application hybride : Ionic, React Native, Flutter, Apache Cordova, ...
  3. développement natif

Plateforme Android

Outils de développement

Application Android

Activité

Interface utilisateur

Plateforme Android

- pile logicielle complète : SE $\rightarrow$ applications de base
- dédiée aux terminaux mobiles :
  - smartphones,
  - tablettes,
  - montres connectées, ...
- sdk pour la création d'applications :
  - outils de dévelopement
  - bibliothèques
  - documentation
- se code en **java** ou **Kotlin**
- site de référence :
  - `http://developer.android.com`

Historique

- octobre 2003 : conception d'un OS mobile par Android Inc. (co-fondé par Andy Rubin)
- août 2005 : rachat d'Android Inc. par Google
- novembre 2007 : création du consortium Open Handset Alliance (Google + industriels) et de l'Android Open SourceProject (AOSP), version beta sous licence Open Source Apache
- octobre 2008 : première version avec téléphone : HTC G1/Dream
- 2014 : version "importante" : 5.X Lollipop
- 2021 : première version de Compose
- actuellement sortie annuelle d'une version majeure
- 2025 : dernière version en date 16

Versions

Les différente versions d'Android :

- port(ai)ent des noms de sucreries (en anglais)
- suivent une logique alphabétique (de A vers Z)
- possèdent un numéro d'API (*API level*) qui s'incrémente

Version	Nom	Sortie	API
5.0 - 5.1	**L**ollipop	11/2014-03/2015	21-22
6.0	**M**arshmallow	10/2015	23
7.0 - 7.1	**N**ougat	08/2016-10/2016	24-25
8.0 - 8.1	**O**reo	08/2017-12/2017	26-27
9.0	**P**ie	2018	28
	...		
13.0	**T**iramisu	2022	33
14.0	**U**pside Down Cake	2023	34
15.0	**V**anilla Ice Cream	2024	35
16.0	**B**aklava	2025	36

Architecture Android

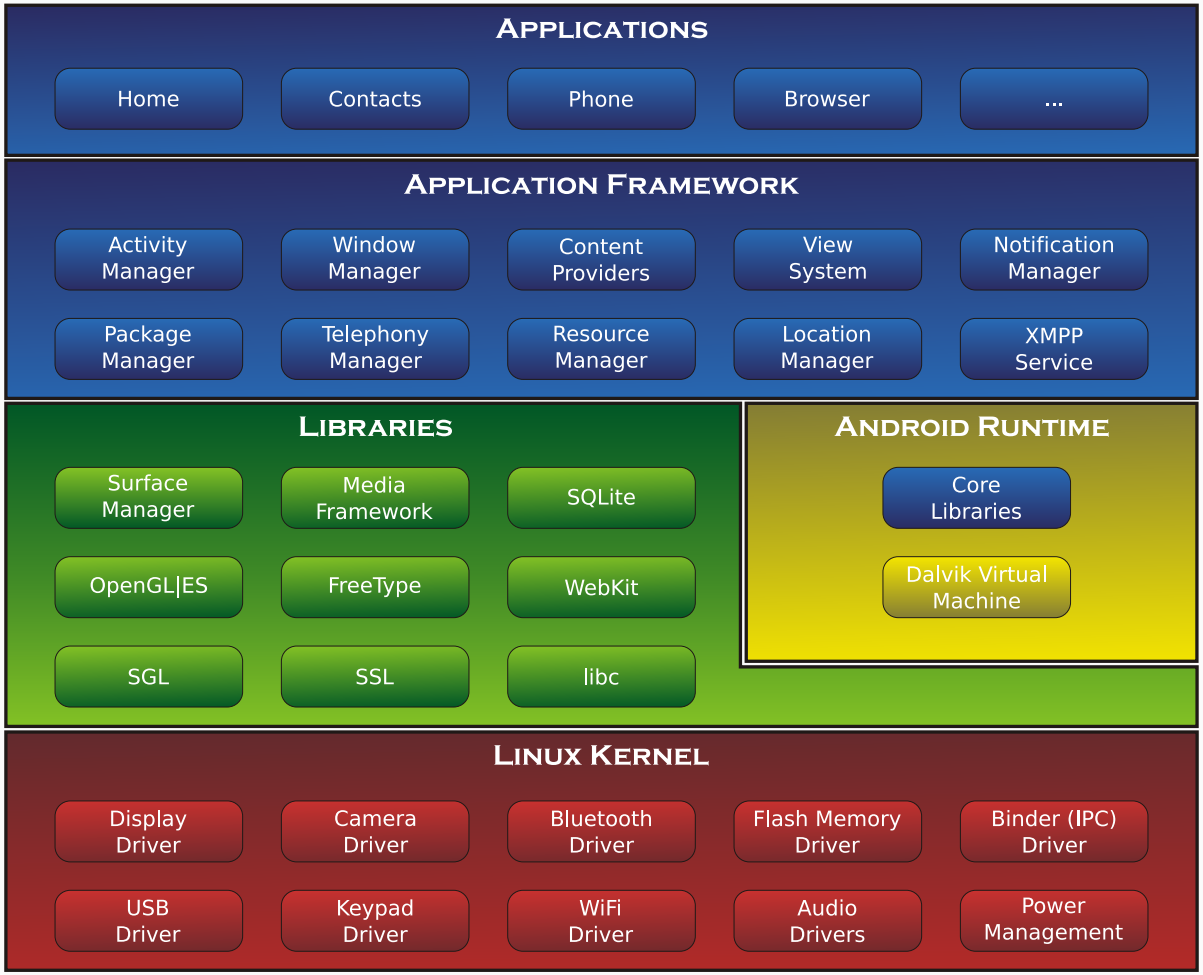
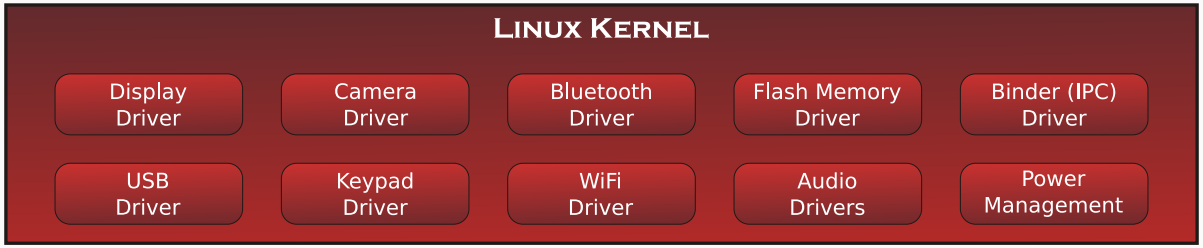


Figure 1 : "Android-System-Architecture" (source : Wikimedia Commons)

Noyau Linux



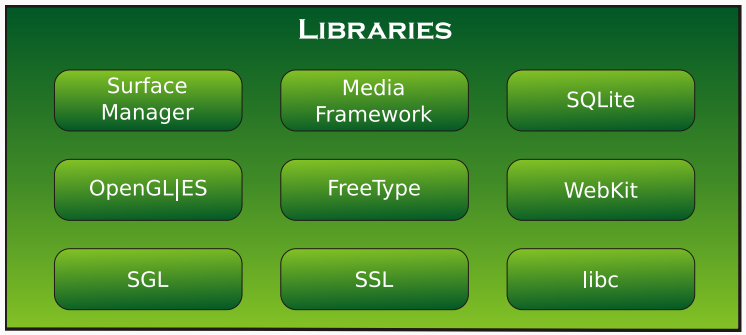
standard :

- gestion de la sécurité
- gestion des processus et de la mémoire
- E/S fichier et réseau
- gestion de périphériques

modifié pour android :

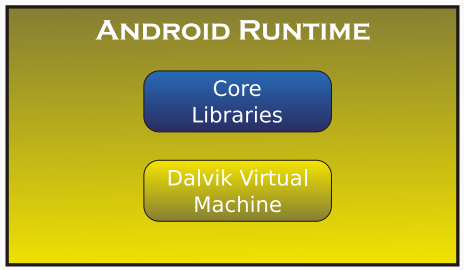
- gestion alimentation
- IPC
- gestion mémoire (*Kernel Memory Killer*)
- ...

Bibliothèques système



- écrites en C et C++
- libc $\rightarrow$ syscall : création process, allocation mémoire
- surface manager : rendu 2D/3D
- media framework : lecture/enregistrement audio/vidéo
- webkit : moteur de rendu de pages web
- openGl : moteur graphique
- sqlite : SGBDR

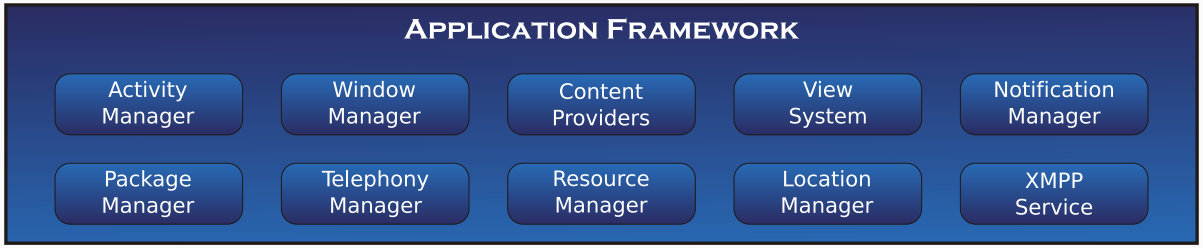
Android Runtime



deux composants principaux :

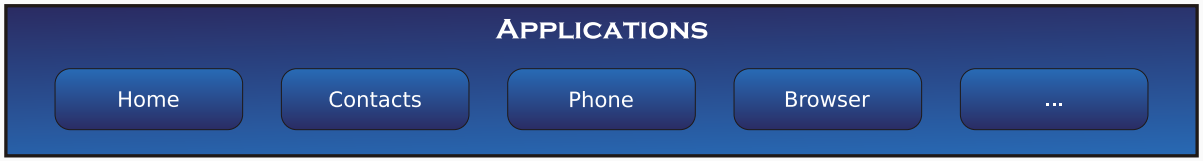
1. bibliothèque java core :
   - classes de base (`java.*`, `javax.*`) : structures de données, IO, réseau, XML...
   - classes spécifiques android `android.*`
   - services internet/web `org.*` (SAX, DOM, JSON)
   - junit
2. machine virtuelle (Dalvik puis ART)

Gestionnaires



- *Activity Manager* : gestion du cycle de vie d'une application
- *Window Manager* : gestion des fenêtres composant une application
- *ContentProvider* : partage de données inter-applications
- *Views System* : composants graphiques (boutons, icônes, listes, ...) pour UI
- *Notification Manager* : permet de placer des informations dans la barre de notification
- *Package Manager* : gestion des applications installées
- *Telephony Manager* : gestion de la téléphonie
- *Ressource Manager* : gestion des ressources (Strings, graphiques, mise en page)
- *Location Manager* : information sur mouvement et position

Applications de base



- écran d'accueil
- contact
- téléphone
- navigateur web
- client mail
- ...

Plateforme Android

Outils de développement

Application Android

Activité

Interface utilisateur

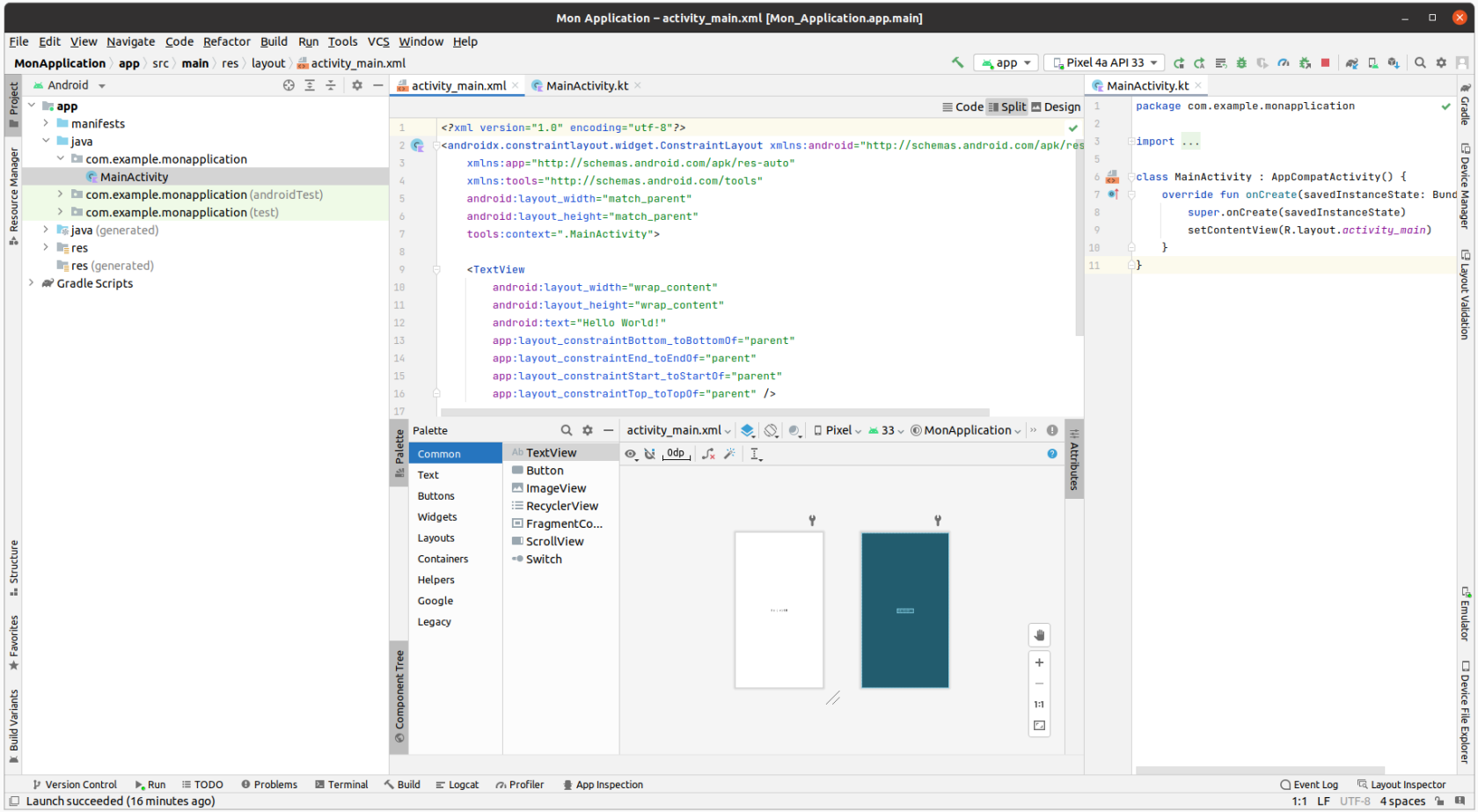
Android SDK (*Software Development Kit*)

ensemble complet d'outils de développement :

- Platform : `android.jar`
  - `.class` : sous ensemble java de base + `android.` + `org.`
  - ressources
- Tools :
  - `sdkmanager` : → Android SDK Manager
  - `avdmanager` : → Android Virtual Device Manager
- `emulator` : émulateur basé sur QEMU
- Platform-tools → `adb`, `sqlite3`,
- Build-tools → `aapt2`, `d8`, `apksigner`, ...

Android Studio

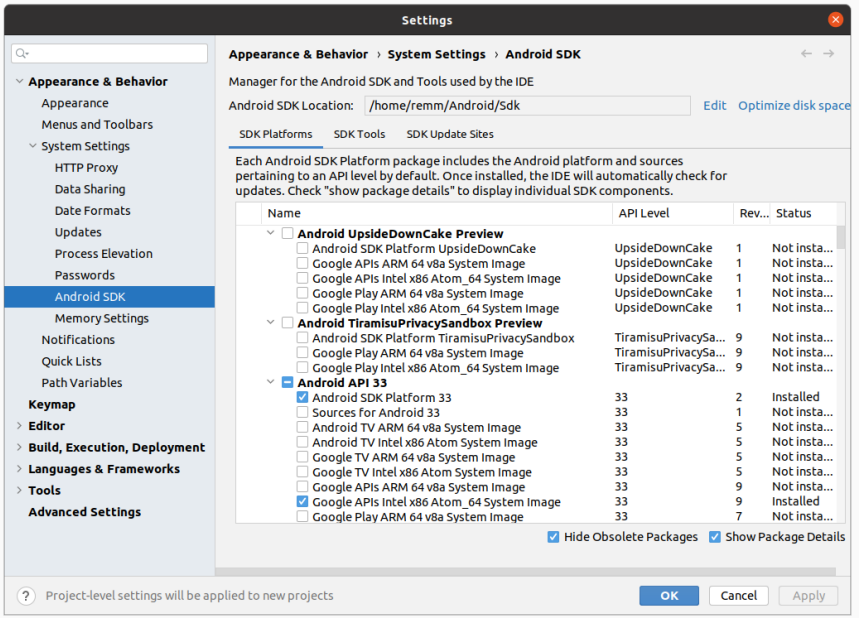
- environnement de développement officiel
- la plupart des outils, par le passé autonomes, y sont intégrés

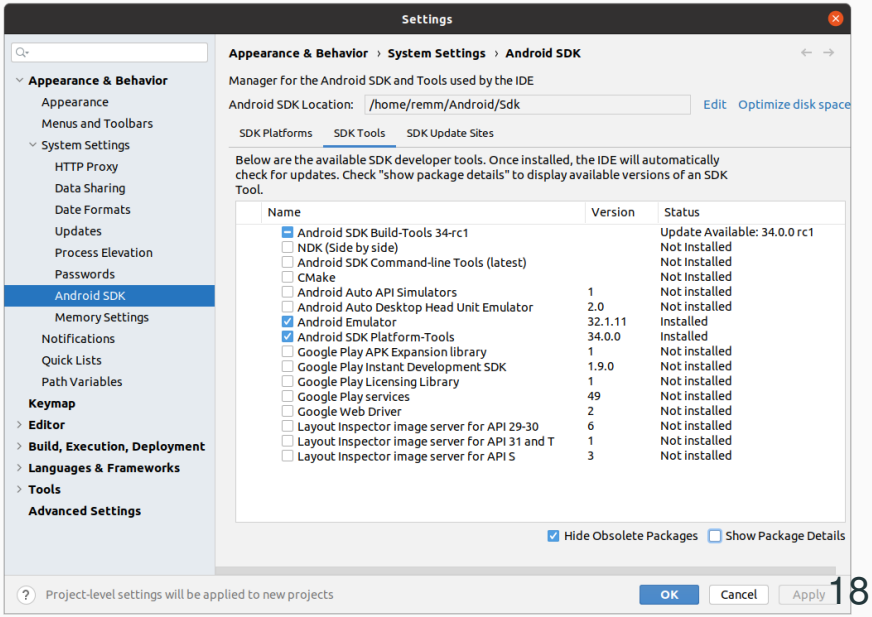


Android SDK Manager

gestion des :

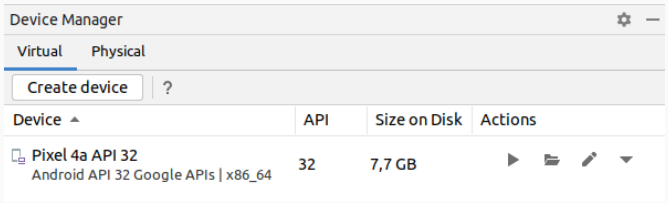
- versions de l'api android installés
- sources des classes android
- images système des émulateurs : `system-images` (avec ou sans apis google)
- outils
- ...

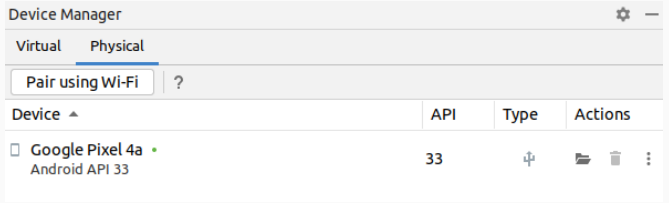




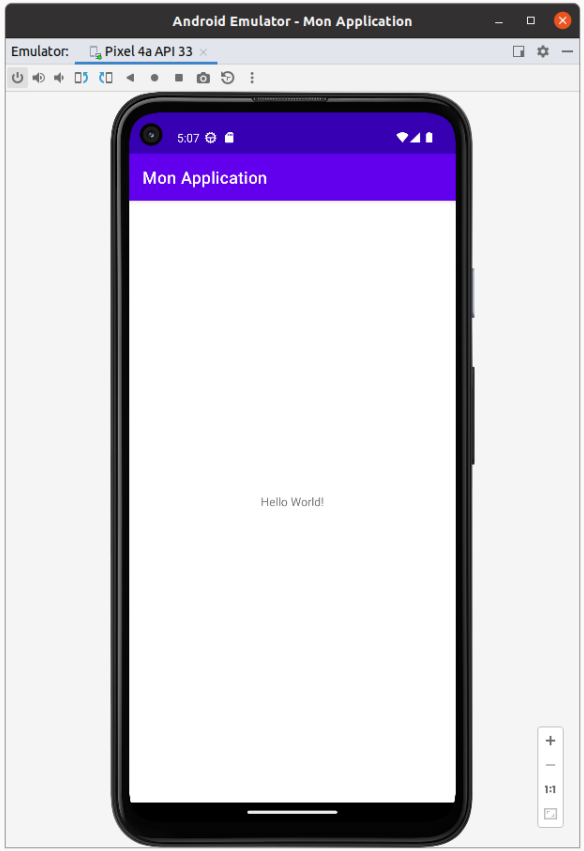
Android Device Manager

- permet de créer des configurations à émuler
- intègre des définitions de terminaux existants
- *Virtual Device* = image système (iso) + configuration matérielle (écran + proc + mem + ...)

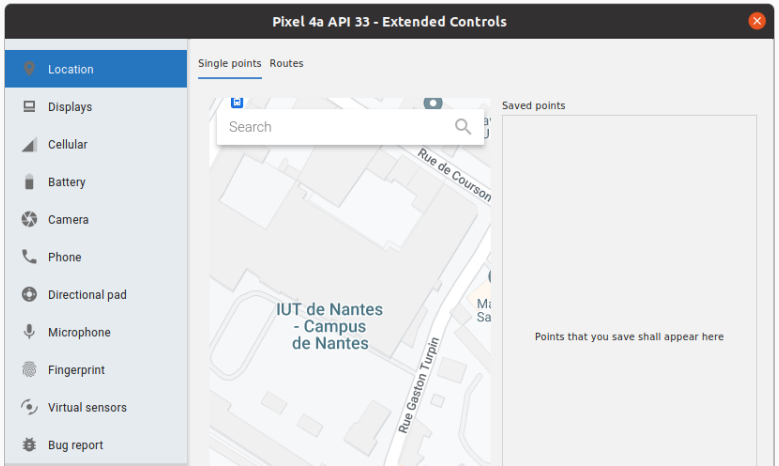




Emulator


Android Emulator - Mon Application
Emulator:
Pixel 4a API 33
5:07
Mon Application
Hello World!


Android Emulator - Mon Application
Emulator:
Pixel 4a API 33
07:07
Jeu. 7 juil.
Gmail
Photos
YouTube


Pixel 4a API 33 - Extended Controls
Location
Displays
Cellular
Battery
Camera
Phone
Directional pad
Microphone
Fingerprint
Virtual sensors
Bug report
Single points
Routes
Search
Saved points
Points that you save shall appear here
IUT de Nantes - Campus de Nantes
Rue de Courson
Rue Gaston Turpin

Dalvik / ART (Android Runtime)

- exécution de bytecode java
- n'interprète pas le bycode java mais des `.dex` (*Dalvik EXecutable*)
- JVM classique : à pile
- Dalvik : basé registre
- java/kotlin $\xrightarrow[\text{compilation}]{}$ .class $\xrightarrow[\text{dx}]{}$ .dex
- Dalvik remplacé depuis Lollipop par ART (*Android RunTime*)
- type de compilation :
  - Dalvik : compilation JIT (Just In Time)
  - ART : compilation (AOT ahead-of-time)
- ART : compilation à l'installation $\Rightarrow$ meilleures performances, économie de batterie, surcout en stockage (30%)

source :

`https://fr.wikipedia.org/wiki/Dalvik_(machine_virtuelle)`

`https://fr.wikipedia.org/wiki/ART_(Android)`

`https://source.android.com/docs/core/runtime`

ADB (Android Debug Bridge)

- à l'origine, utilitaire en ligne de commande
- maintenant, plutôt via Android Studio
- fonctionnement client/serveur avec un émulateur lancé
- utilisation directe compliquée mais toujours possible
- exemples :
  - lister les périphériques :

```
adb devices
List of devices attached
0698061e0036d349 device
emulator-5554     device
```

  - copie(push)/lecture(pull) de fichier :

```
adb push monfichier.mp3 /sdcard/Music
```

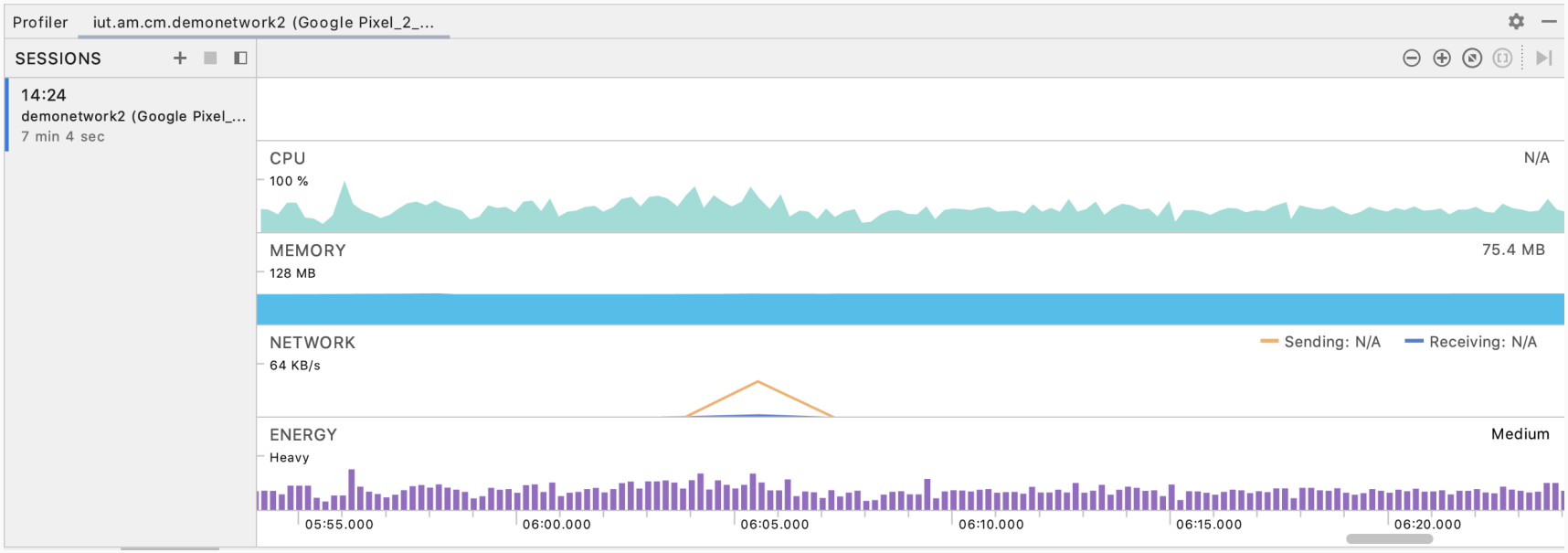
  - (de)installation (install/uninstall) d'une application :

```
adb install monApp.apk
```

  - shell direct possible

```
adb shell
$ sqlite3 /data/data/com.example.app/databases/rssitems
```

Android Profiler



Plateforme Android

Outils de développement

Application Android

Activité

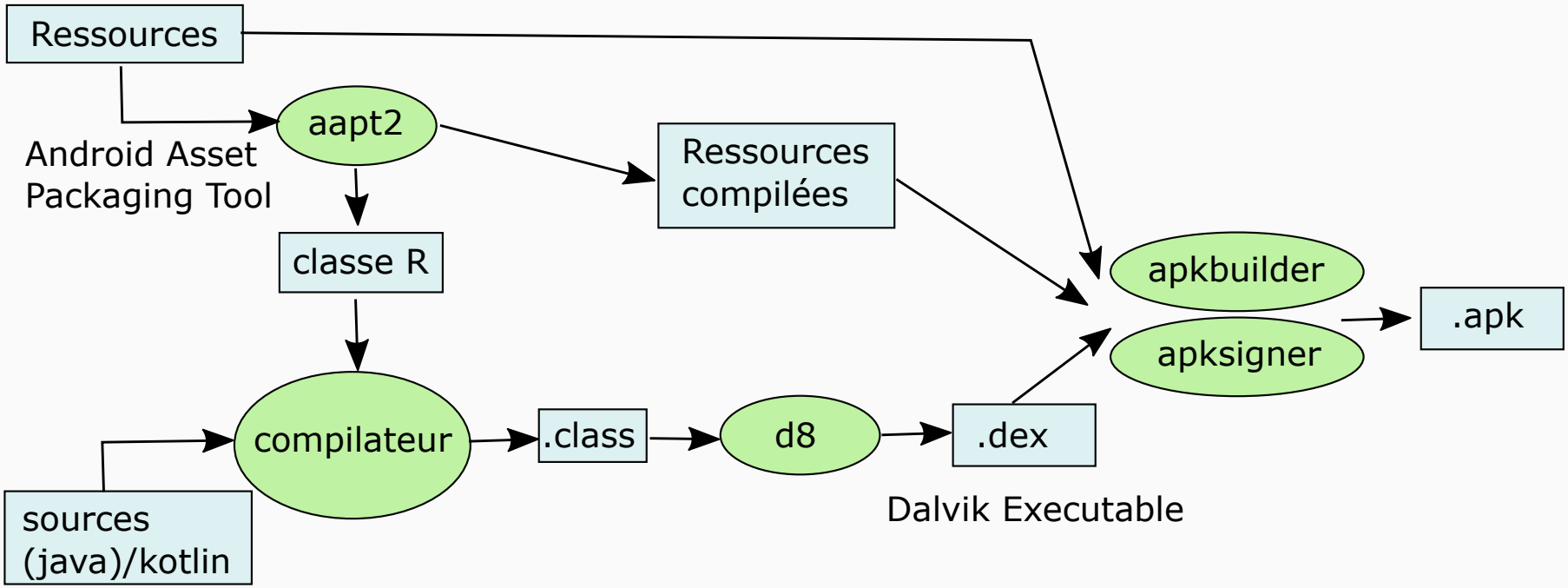
Interface utilisateur

Application Android

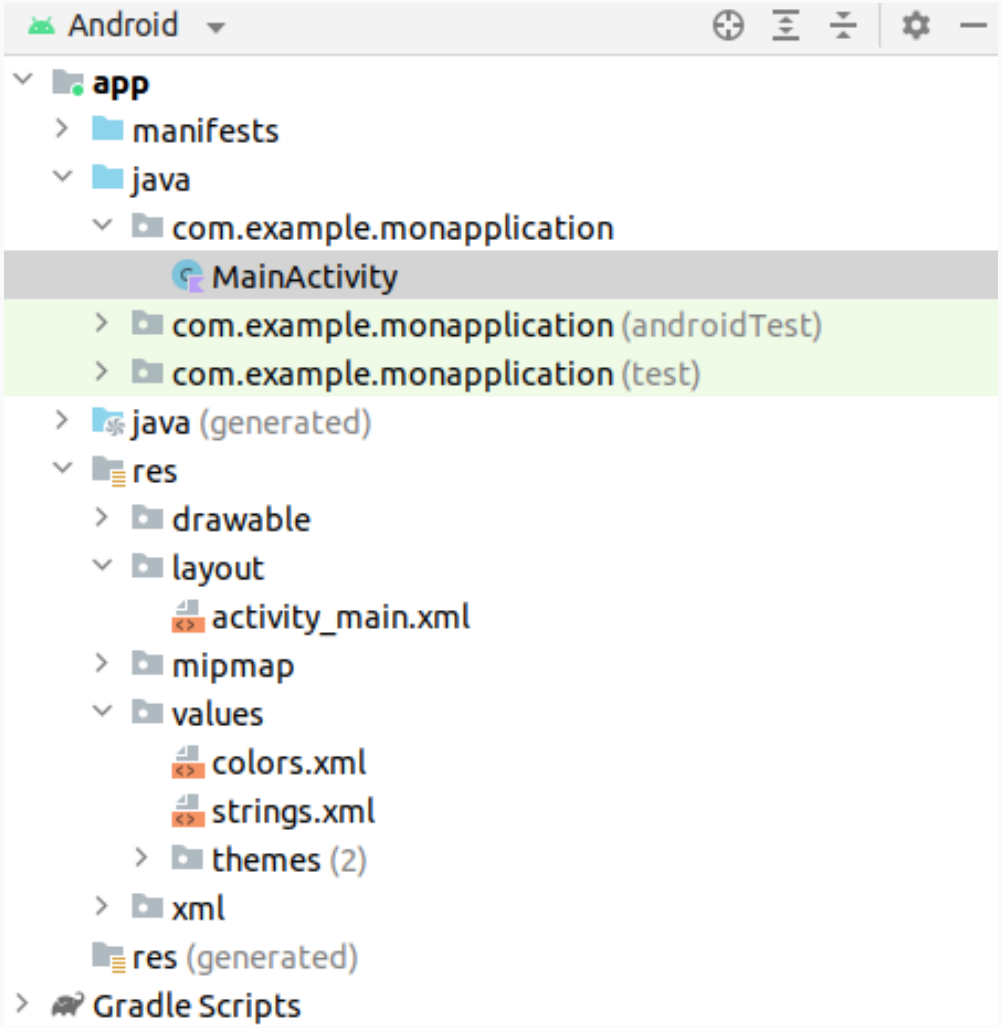
application android :

- écrite en kotlin (java à l'origine)
- déployée sous forme d'un fichier apk (*Android PacKage*)
- chaque application s'exécute sous l'identité d'un user différent sur le système linux sous jacent
- constituants :
  1. fichiers XML :
     - `AndroidManifest.xml` : description de application (version de l'API utilisée, activité principale, etc)
     - `*.xml` : description des ressources (texte, image, etc)
  2. code kotlin/java : spécification de la partie dynamique de l'application
  3. fichiers de ressources (image, son, ...)

Production d'une application (simplifié)



Structure d'une application



AndroidManifest.xml (simplifié)

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.MonApplication">
        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:label="@string/app_name"
            android:theme="@style/Theme.MonApplication">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
```

Ressources

- images, fichiers de configuration, description des interfaces graphiques
- séparées du code source
- placées dans un répertoire `/res` de l'application
- incorporées au binaire de l'application
  - `drawables` : images (i.e. fichiers png, gif, ...) ou des fichiers XML.
  - ~~`layout` : définitions en XML des interfaces graphiques des écrans ou fragments d'écran~~
  - `menu` : fichiers XML décrivant les menus de l'application
  - `font` : fichiers XML ou (par exemple) `.ttf`
  - `values` : définitions de chaînes, de couleurs, de dimensions, ... en XML (fichier général `values.xml` ou spécialisés `strings.xml`, `colors.xml`, `styles.xml`, ...)

source :

`https://developer.android.com/guide/topics/resources`

Ressources : exemple

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string name="nom">Démonstration</string>
    <string name="hello_world">Hello world!</string>

    <string-array name="liste">
        <item>IOS</item>
        <item>Android</item>
        <item>Windows Phone</item>
    </string-array>

    <color name="bleu">#ff0000ff</color>

    <dimen name="unemarge">10dp</dimen>

</resources>
```

Ressources alternatives

- une application peut proposer des ressources alternatives
- par la création d'un répertoire dans `res/`.
- exemple :

```
res/
  drawable/
      icon.png
      background.png
  drawable-hdpi/
      icon.png
      background.png
```

- en fonction de $\neq$ critères :
  - tailles d'écran : `small`, `normal`, `large` et `extra-large`.
  - densité d'écran : `ldpi`, `mdpi`, `hpdi`, `xdpi`, `xxdpi`, ... en fonction de la capacité en dpi (*dots per inch*) de l'écran
  - langue : `en`, `fr`
  - orientation de l'écran `port`, `land`
  - pays et/ou opérateur : MCC (*mobile country code*) et MNC (*mobile network code*). ex : `mcc208-mnc00` France-orange
  -

Classe R

- `R` classe générée par `aapt` permet d'accéder aux ressources
- une classe interne par type de ressource (ex : `R.string`)
- une constante entière par ressource (ex : `R.string.deux`.)
- cette constante est un ID qui permettra de retrouver la ressource

```
public final class R {
    public static final class string {
        public static final int app_name=0x7f060000;
        public static final int deux=0x7f060001;
        public static final int trois=0x7f060002;
        public static final int un=0x7f060003;
    }
}
```

Composants d'une application

1. *Activity* : un écran doté d'une interface graphique
2. *Service* : exécutés en arrière plan, permettent des opérations longue ou une interaction avec des processus distant
3. *BroadcastReceiver* : écoute et réaction à des événements (`Intent`)
4. *Content Provider* : stockage et partage de données entre applications

Plateforme Android

Outils de développement

Application Android

Activité

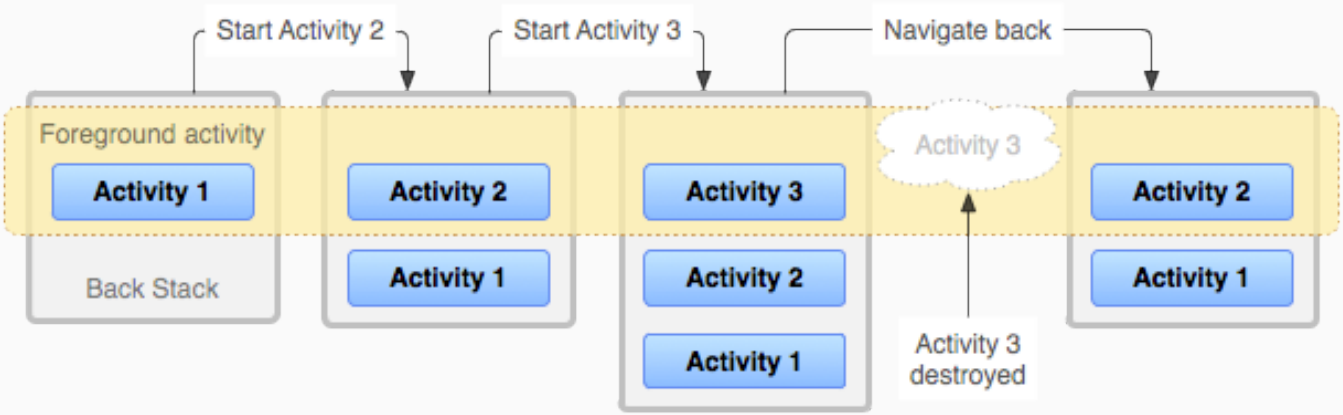
Interface utilisateur

Activité

- application Android = des écrans
- un écran = une Activité (`Activity`)
- exemples :
  - visualiser un mail
  - afficher une fenêtre de login
  - ...
- une Activité peut en lancer une autre
- les Activités peuvent être interrompues, reprises, tuées, ...

BackStack

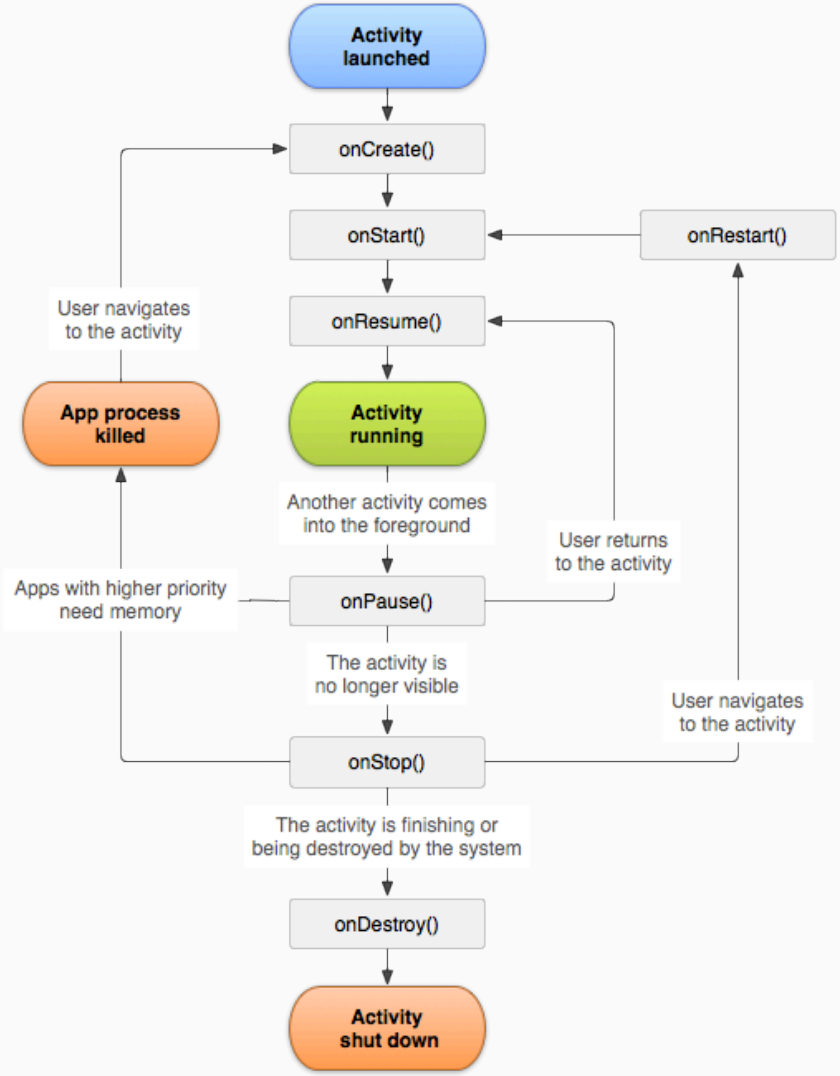
- les activités sont gérés à l'aide d'une pile
- une activité peut lancer une autre activité de la même application (ex : liste de mails $\rightarrow$ détail)
- une activité peut lancer une autre activité d'une autre application (ex : appareils photo $\rightarrow$ composeur de mms).
- back, home, plus de ressources ?



source :

`https://developer.android.com/guide/components/activities/tasks-and-back-stack.html`

Cycle de vie d'une activité



- événements :
  - action de l'utilisateur : `back`, `home`
  - événements systèmes
- visibilité :
  - visible et au premier plan : `onResume()-onPause()`
  - visible : `onStart()-onStop()`

`https://developer.android.com/guide/components/activities/activity-lifecycle.html`

Plateforme Android

Outils de développement

Application Android

Activité

Interface utilisateur

Interface utilisateur

- appelée *UI*
- deux manières :
  1. traditionnelle : *Android View*
  2. moderne : *Compose*

UI : *Android View*

- une arborescence de composants (`View`) et `ViewGroup`
- positionnés à l'aide de gestionnaire de placement (*layout*)
- définie dans un fichier XML
- les vues possèdent un état interne qui doit être modifié programmatiquement
- avec de la gestion d'événements en kotlin

UI *Android View* : XML

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity" >

    <TextView
        android:id="@+id/textView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_centerInParent="true"
        android:text="Test" />
</RelativeLayout>
```

UI *Android View* : association avec l'activité

```
import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
    }
}
```

UI *Android View* : gestion des événements

```
class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val bouton = findViewById<Button>(R.id.button)
        bouton.setOnClickListener { action() }
    }

    private fun action() {
        Toast.makeText(this, "Ça marche !",
            Toast.LENGTH_SHORT).show()
    }
}
```

UI *Android View*

problèmes :

- gestion des états : complexité
- beaucoup de code répétitif : `findViewById()`
- performances : hiérarchie des vues lourde, source de ralentissements

concurrence :

- frameworks déclaratifs : React ou SwiftUI pour iOS

UI : *Compose*

- approche déclarative : **Quoi** plutôt que **Comment**
- interface définie en code par des fonctions **modulables** qui :
  - collectent des données
  - émettent des éléments d'UI
- les composants ne possèdent pas d'état interne
- l'état est géré à part
- un modification de l'état entraine le re-dessin de l'interface

UI : fonction composable/modulable

- annotée par `@Composable`
- signature : `@Composable () -> Unit`
- appelé composable et en fait est une instance de `Layout`
- peut en appeler d'autres

```
@Composable
fun Bonjour(name: String, modifier: Modifier = Modifier) {
    Text(
        text = "Hello $name!",
        modifier = modifier
    )
}
```

UI *Compose* : association avec l'activité

```
import android.os.Bundle
import androidx.activity.ComponentActivity
// ...

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            SimpleTheme {
                Bonjour(
                        name = "Android"
                    )
                }
            }
        }
    }
```

Layout : Kotlin Compose

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            CmComposeBasicTheme {
                Surface(
                    modifier = Modifier.fillMaxSize(),
                    color = MaterialTheme.colors.background
                ) {
                    TexteMilieu("Test")
                }
            }
        }
    }
}

@Composable
fun TexteMilieu(name: String) {
    Column(horizontalAlignment = Alignment.CenterHorizontally,
            verticalArrangement = Arrangement.Center) {
            Text(modifier = Modifier.padding(16.dp),
                text = "$name!")
        }
}
```

UI : Composables de Base

- `Text` : afficher du texte (remplace TextView).
- `Button` : bouton cliquable (peut contenir texte ou icône).
- `Image` : afficher des ressources graphiques ou des icônes.
- `TextField` : saisie de texte par l'utilisateur (remplace EditText).
- `Icon` : afficher des icônes vectorielles.

```
@Composable
fun Bonjour(name: String, modifier: Modifier = Modifier) {
    Text(
        text = "Hello $name!",
        modifier = modifier
    )
}
```

UI : Composables de Structure

qu'on pourrait qualifier de conteneurs :

- `Column` : aligne ses enfants verticalement
- `Row` : aligne ses enfants horizontalement
- `Box` : superpose ses enfants
- `Scaffold` : structure générique d'application (Barre en haut et en bas, place pour le contenu principal, bouton flottant, menu, ...)

```
Row(
        modifier = modifier,
        verticalAlignment = Alignment.CenterVertically
    ) {
        Text(
            text = "Un"
        )
        Text(
            text = "Deux"
        )
    }
```

Alignment/Arrangement

- pour les `Column` et les `Row`
- permet de gérer la dispositions des composants enfants dans les conteneurs parents
- `Arrangement` : répartition sur l'axe principal
- `Alignment` : répartition sur l'axe secondaire (perpendiculaire au flux)

Column

- `Column` : axe principal = vertical, axe secondaire = horizontal
- `VerticalArrangement` : espace entre les éléments (`Top`, `Center`, `Bottom`, `SpaceBetween`, etc.).
- `HorizontalAlignment` : Gère le placement latéral (`Start`, `CenterHorizontally`, `End`).

```
Column(
    modifier = Modifier.fillMaxSize(),
    verticalArrangement = Arrangement.SpaceEvenly,
    horizontalAlignment = Alignment.CenterHorizontally
) {
    Text("Élément 1")
    Text("Élément 2")
}
```

Row

- `Row` : axe principal = horizontal, axe secondaire = vertical
- `HorizontalArrangement` : espace entre les éléments (`Start`, `Center`, `End`, `SpaceAround`, ...).
- `VerticalAlignment` : gère le placement vertical (`Top`, `CenterVertically`, `Bottom`).

```
Row(
    modifier = Modifier.fillMaxWidth(),
    horizontalArrangement = Arrangement.SpaceBetween,
    verticalAlignment = Alignment.CenterVertically
) {
    Icon(Icons.Default.Menu, contentDescription = null)
    Text("Ma super Appli !")
    Icon(Icons.Default.Settings, contentDescription = null)
}
```

Box

- `Box` : empile les éléments les uns sur les autres (Z-axis).
- `contentAlignment` : aligne tous les enfants par défaut (`TopStart`, `Center`, `BottomEnd`, ...).
- `Modifier.align` : peut être utilisé sur un enfant spécifique pour surcharger l'alignement global.

```
Box(
        modifier = Modifier.size(120.dp),
        contentAlignment = Alignment.Center
    ) {
        Image(
            painterResource(id = R.drawable.openlogo_nd),
            contentDescription = null
        )
        Text(
            "Hello $name!", color = Color.Black,
            fontWeight = FontWeight.Bold,
            fontFamily = FontFamily.Monospace
        )
    }
```

UI : arguments vs `Modifier`

- l'argument définit ce que le composable est (ses données intrinsèques).
  ex : texte d'un `Text`, action `onClick` d'un `Button`, ...
- le `Modifier` définit comment le composable se comporte vis-à-vis de son environnement
  ex : taille, marge, arrière-plan, ou s'il est cliquable (`clickable`)

UI : `Text`

arguments :

- `text` : La chaîne de caractères à afficher.
- `style` : typographie, qui peut être celle du thème (ex : MaterialTheme.typography.bodyMedium).
- `color` : changer la couleur du texte.
- `fontWeight` : épaisseur (ex : `FontWeight.Bold`).
- `fontStyle` : style (ex : `FontStyle.Italic`)
- `fontSize` : taille
- `fontFamily` : type (ex : `FontFamily.Monospace`)
- `textAlign` : Pour aligner le texte à l'intérieur de son propre bloc.

UI : Autres

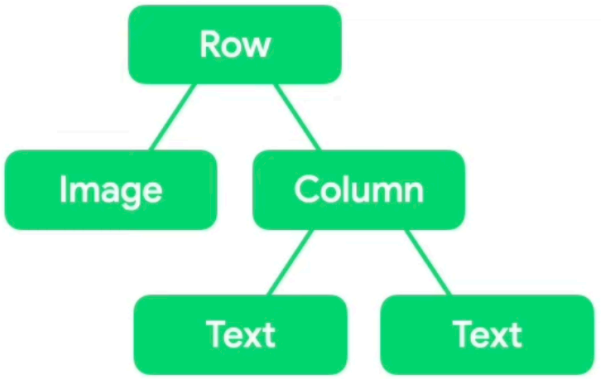
TODO

UI : principe de rendu

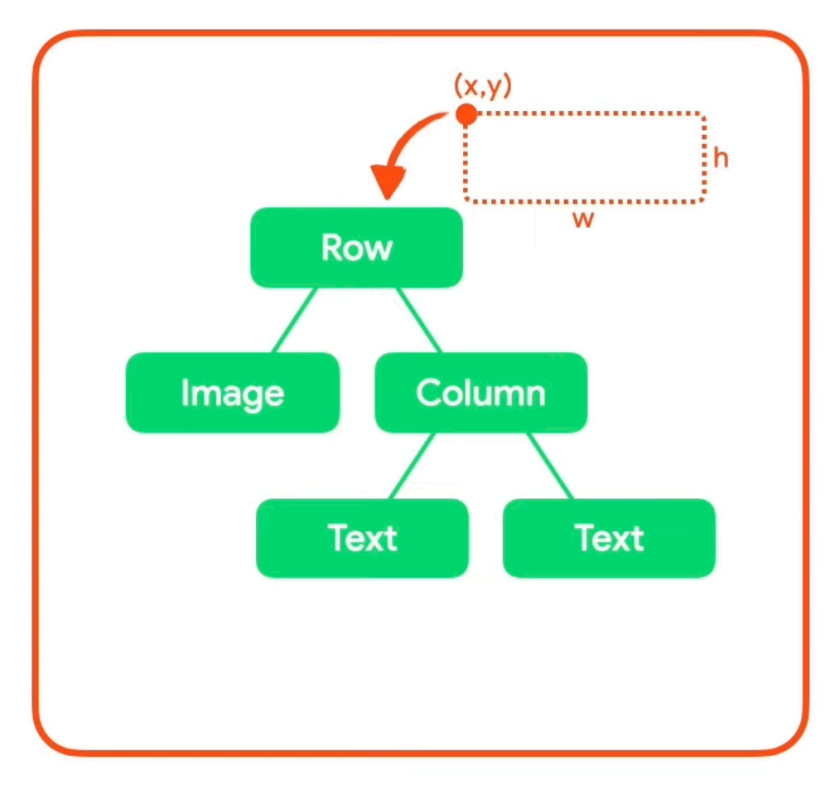
- trois phases :
  1. (re)composition : production d'une arborescence de composants
  2. positionnement (algo récursif) :
     - 2.1 mesure des enfants
     - 2.2 détermination de sa propre taille
     - 2.3 positionnement des enfants
  3. dessin
- pour arriver à faire le positionnement, chaque noeud ne doit être visité qu'une seule fois

UI : Composition

```
Row {
    Image(..)
    Column {
        Text(..)
        Text(..)
    }
}
```



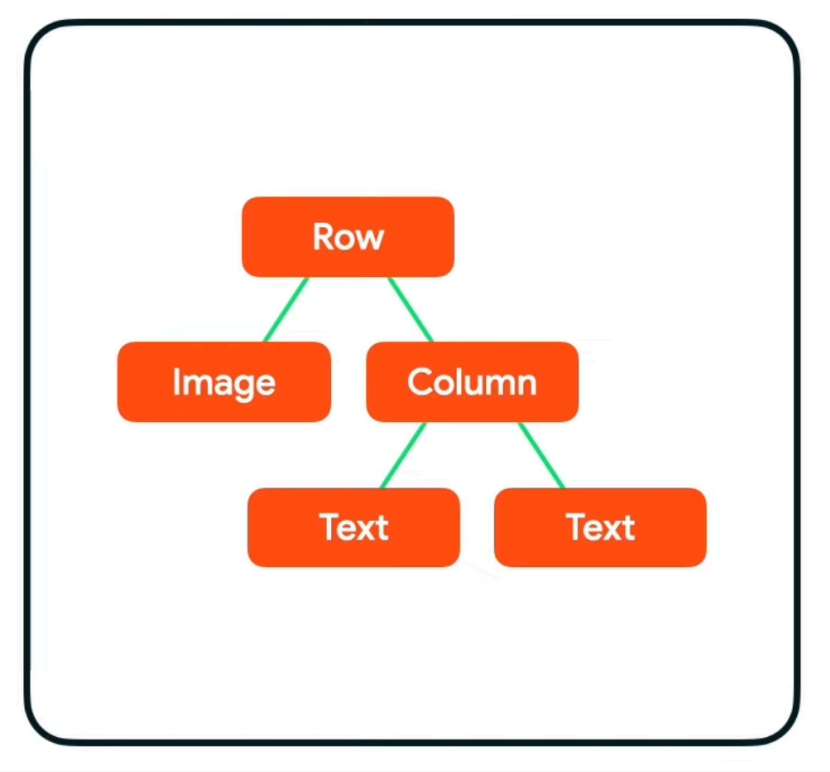
UI : Placement



UI : Contraintes

- contraintes transmises via l'objet `Constraints`.
- structure de quatre valeurs : `minWidth` / `maxWidth`, `minHeight` / `maxHeigh`
- TODO

UI : Dessin



Accès aux ressources : Compose

- pour utiliser une string définie dans un fichier de ressources dans une interface graphique
- syntaxe :
  `[<package_name>.]R.<resource_type>.<resource_name>`
- exemple :

`/res/values/strings.xml`

```
<resources>
    <string name="hello_world">Hello world!</string>
</resources>
```

```
Text(
    text = stringResource(R.string.hello_world),
    modifier = modifier
)
```

Thème et Style

- thème = configuration globale

/ui/theme/Theme.kt

```
@Composable
fun MonApplicationTheme(
    darkTheme: Boolean = isSystemInDarkTheme(),
    dynamicColor: Boolean = false, // CONSEILLÉ
    content: @Composable () -> Unit
) {
    val colorScheme = if (darkTheme) DarkColorScheme
                      else LightColorScheme

    MaterialTheme(
        colorScheme = colorScheme,
        typography = Typography,
        content = content
    )
}
```

Thème et Style

trois catégories principales :

1. Couleurs : définition des couleurs (`Primary`, , ...)

   `/ui/theme/Color.kt`

   ```
   val primaryLight = Color(0xFF145F42)
   val onPrimaryLight = Color(0xFFFFFFFF)
   ```

   et

   `/ui/theme/Theme.kt`

   ```
   private val lightScheme = lightColorScheme(
     primary = primaryLight,
     onPrimary = onPrimaryLight,
     ...
   ```

Thème et Style

2. Typographie : police et style des texte (body, headline, ...)

`/ui/theme/Type.kt`

```
    val displayFontFamily = FontFamily(
    Font(
        googleFont = GoogleFont("Esteban"),
        fontProvider = provider,
    )
    bodyMedium = baseline.bodyMedium.copy
      fontFamily = bodyFontFamily)
)
```

3. Forme (*Shape*) : formes des composants, + ou - arrondis

- outil de création :
  `https://material-foundation.github.io/material-theme-builder`

Thème et Style

- style = attributs définissant l'aspect visuel d'un composant (ex : un bouton)
- style, comme en css = dimensions, marges, couleurs, polices, ...
- style donné par :
  1. le Thème (`MaterialTheme`) : propriétés intrinsèques (couleurs, style de police, ...)
  2. les `Modifiers` : instructions de mise en page (taille, marge, position, couleur de fond, ...)

```
Text(
    text = "Hello $name!",
    modifier = modifier.background(
          MaterialTheme.colorScheme.background)
    color = Color.Red //
    modifier = modifier.border(
          BorderStroke(2.dp, Color.Blue))
)
```

Unité de dimensions

- utilisation d'unités indépendantes de la résolution de l'écran
- en particulier :
  1. `dp` *density-independent pixels* : correspond à une unité de pixel virtuelle où 160dp $\simeq$ 1 pouce (2,54 cm). Adapté par le système en fonction de la densité de pixel des écrans
  2. `sp` *scalable pixels* idem `dp` mais redimensionnée selon les préférences de l'utilisateur pour la taille du texte

```
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp

Column(modifier = Modifier.padding(16.dp)) {
    Text(
        text = "Titre important",
        fontSize = 20.sp
    )
    Spacer(modifier = Modifier.height(8.dp))
    Button(modifier = Modifier.height(48.dp)) {
        Text("Valider")
    }
}
```

UI : état

- l'état est observé par compose
- une modification de l'état entraine la modification de UI
- l'état est défini par une collection de variables de type(s) défini(s) :
  - par défaut : `State<T>` ou `MutableState<T>` (sous interface de `State<T>`)
  - d'autres types sont possibles comme `Flow` et `LiveData` grâce à des conversions
- pour que l'état ne soit pas réinitialisé il est mémorisé dans la composition $\rightarrow$ `remember`

UI : `State<T>`

- gestion de l'état la plus basique
- conseillé pour l'état local à un composant (ex : le texte d'un champ de saisie)
- ∃ plusieurs façons de déclarer un objet `MutableState` :

```
val mutableState = remember { mutableStateOf(default) }
var value by remember { mutableStateOf(default) }
val (value, setValue) = remember { mutableStateOf(default) }
```

- syntaxe by :

```
import androidx.compose.runtime.getValue
import androidx.compose.runtime.setValue
```

- en cas de changement de configuration, l'état est perdu → `rememberSaveable`

UI : State<T>

```
@Composable
fun EcranCompteur2( modifier: Modifier = Modifier) {

    var compteur = remember { mutableStateOf(0) }

    Column {
        Text("Valeur : ${compteur.value}",style = MaterialTheme.typogr
        Button(onClick = { compteur.value++ }) {
            Text("Incrémente")
        }
    }
}
```

Syntaxe avec by : accès direct à compteur

```
@Composable
fun EcranCompteur3( modifier: Modifier = Modifier) {

    var compteur by remember { mutableStateOf(0) }

    Column {
        Text("Valeur : $compteur",style = MaterialTheme.typography.hea
        Button(onClick = { compteur++ }) {
            Text("Incrémente")
        }
    }
}
```

UI : Hissage d'état (*state hoisting)*

- deux types de composables : sans état (*stateless*) / avec état (*stateful*)
- composable avec remember = composable avec état
- moins facile à tester, moins réutilisable
- pour faire un composant sans état, hissage d'état
- hissage d'état = remonter l'état vers l'appelant d'un composable
- modèle pour hissage d'état = remplacer la variable d'état par deux paramètres :
  - `value: T` : valeur à afficher
  - `onValueChange: (T) -> Unit` : événement où T est la nouvelle valeur

UI : Hissage d'état (*state hoisting)*

```
@Composable
fun EcranCompteur4( modifier: Modifier = Modifier) {
    var compteur by remember { mutableStateOf(0) }

    ComposantCompteur(
        value = compteur,
        onValueChange = {compteur++}
    )
}

@Composable
fun ComposantCompteur(value : Int, onValueChange : () -> Unit) {
    Column {
        Text("Valeur : $value",style = MaterialTheme.typography.headli
        Button(onClick = onValueChange) {
            Text("Incrémente")
        }
    }
}
```

UI : ViewModel

- possède un cycle de vie différent de l'activité, conservé même en cas de recréation de l'activité
- utilisé comme conteneur d'état
- fait le lien entre la couche de données et l'UI

UI : State & ViewModel

- plus simple

```
class DemoViewModel : ViewModel() {
    private val _compteur = mutableStateOf<Int>(0)
    val compteur : State<Int>
        get() = _compteur

    fun incremente() {
        _compteur.value++
    }
}
```

UI : State & ViewModel

```
@Composable
fun EcranCompteur(viewModel: DemoViewModel = viewModel()) {
    // déclaré val ; la modification est faite dans le viewModel
    val compteur by viewModel.compteur

    ComposantCompteur(
            value = compteur,
            onValueChange = { viewModel.incremente() }
        )
```

UI : State & ViewModel (version by)

- plus simple

```
class DemoViewModel : ViewModel() {
    var compteur by mutableStateOf<Int>(0)
        private set

    fun incremente() {
        compteur++
    }
}
```

UI : StateFlow & ViewModel

- complexité accrue
- de + en + utilisé
- compatible avec la couche de donnée

```
class DemoViewModel : ViewModel() {

    private val _compteur = MutableStateFlow(0)
    val compteur : StateFlow<Int> = _compteur.asStateFlow()

    fun incremente() {
        _compteur.value++
    }
}
```

UI : StateFlow & ViewModel

```
@Composable
fun EcranCompteur(viewModel: DemoViewModel = viewModel()) {
    val compteur by viewModel.compteur.collectAsStateWithLifecycle()

    ComposantCompteur(
        value = compteur,
        onValueChange = { viewModel.incremente() }
    )
}
```

UI : State[Flow] & ViewModel

- Pour un état plus complexe :

```
data class UserState(
    val name: String,
    val isAdmin: Boolean
  )
```

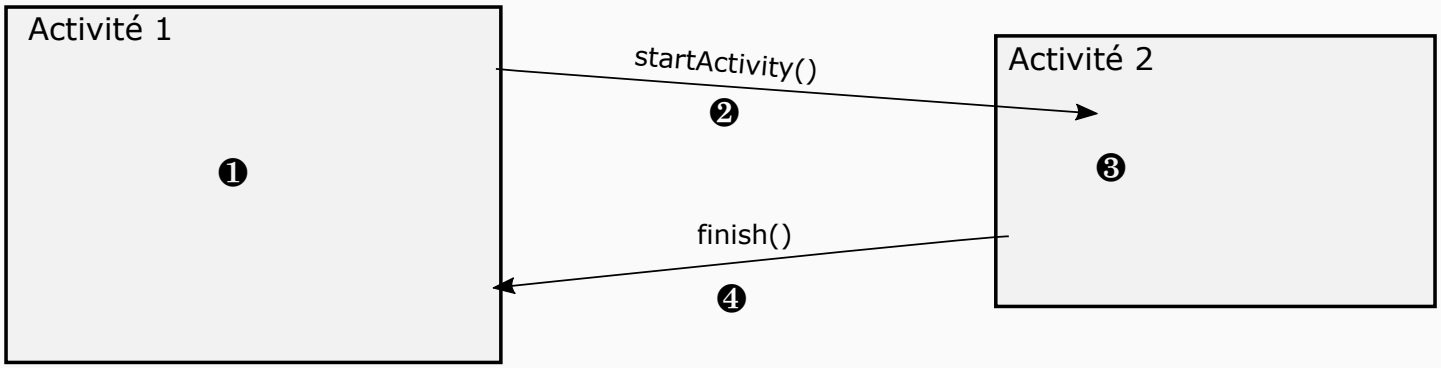
mise à jour :

- Sate : réaffectation de la variable...
- StateFlow : méthode `update` atomique pour éviter les problèmes lié à la concurrence
- dans les deux cas, on utilise `.copy` sur la data class

Communication entre activité : `Intent`

- nécessité de communiquer entre activités
- compliqué entre applications différentes (pas le même espace mémoire)
- activités (`Activity`), services (`Service`) et récepteurs de diffusion (`BroadcastReceiver`) utilisent des `android.content.Intent`
- `Intent` :
  - permet de spécifier (de manière souple) une opération à exécuter : prendre une photo, envoyer un sms, ...
  - construite par un composant "demandeur"
  - reçue par une activité qui réalise la demande
  - exprimée de manière :
    1. **explicite** en désignant l'activité réceptrice
    2. **implicite** en décrivant l'opération souhaitée

`Intent` : Changement d'activité



- dans Activité 1 :

  `startActivity (Intent intent)`

- exemple :

```
val context = LocalContext.current
val versAutre = Intent(context, SecondActivity::class.java)
startActivity(versAutre)
```

- pour revenir dans Activité 1 depuis Activité 2 :

  `finish()` ou *back*

- attention l'activité se déclare dans le manifest

```
<activity android:name=".SecondActivity" > ... </activity>
```

- sinon....`android.content.ActivityNotFoundException`

Intent

- objet comportant de nombreuses propriétés :
  1. action : ce qu'on souhaite du destinataire
  2. donnée : données sur lesquelles réaliser l'action
  3. type (mime) :
  4. catégorie
  5. composant (cible) : demande d'une activité particulière $\rightarrow$ explicite
  6. extra : passer des données à l'aide d'une map clé/valeur
  7. flags
- un `Intent` est créé et transmis d'une activité à une autre pour communiquer
- toutes les propriétés ne sont pas forcement à renseigner

`http://developer.android.com/reference/android/content/Intent.html`

`Intent` : structure 1/2

- action (*action*) : chaîne représentant l'action à réaliser

  `ACTION_MAIN`, `ACTION_VIEW`, `ACTION_DIAL`, `ACTION_DELETE`, ...

```
val newInt = Intent(Intent.ACTION_DIAL)
// ou
val newInt = Intent()
newInt.action = Intent.ACTION_DIAL
```

- donnée (*data*) : donnée sur laquelle agir, formatée selon une URI et/ou type MIME

```
val newInt = Intent(
    Intent.ACTION_DIAL,
    Uri.parse("tel:+33240503060")
)
// ou
val newInt = Intent(Intent.ACTION_DIAL)
newInt.data = Uri.parse("tel:+33240503060")
```

`Intent` : structure 2/2

- catégorie (*category*) : informations supplémentaires sur le composant traitant l'Intent. `CATEGORY_BROWSABLE`, `CATEGORY_LAUNCHER`
- type (*type*) : inféré par les données ou type MIME

```
uneIntent.type = "image/jpeg"
```

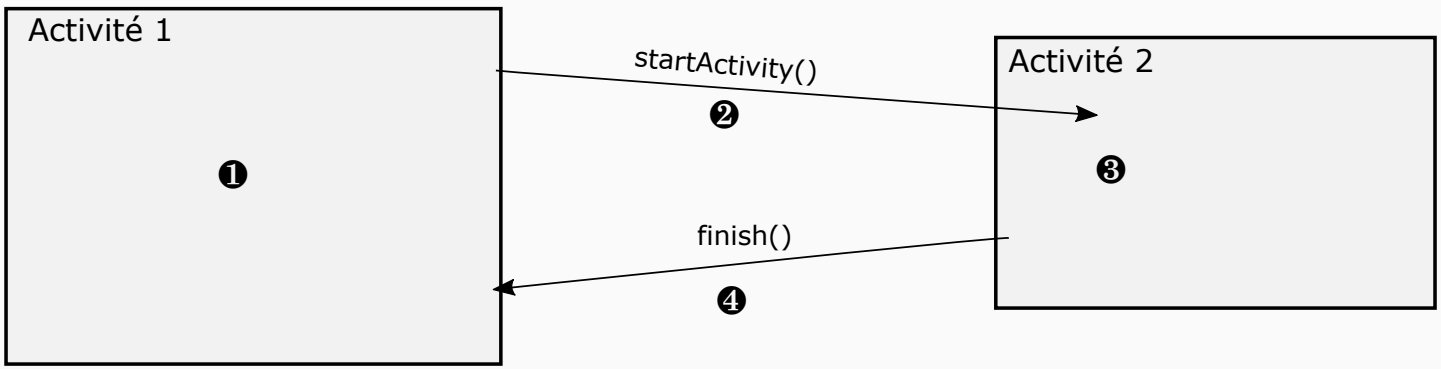
- composant (`component`) : en cas d'Intent **explicite** , classe de l'activité à lancer. Utilisation du constructeur

```
Intent(Context packageContext, Class<?> cls)
```

- extras (*extras*) : ajout de données sous forme de clé/valeur

```
unIntent.putExtra(Intent.EXTRA_SUBJECT,
"Une démo extra !!")
```

Intent explicite



explicite :

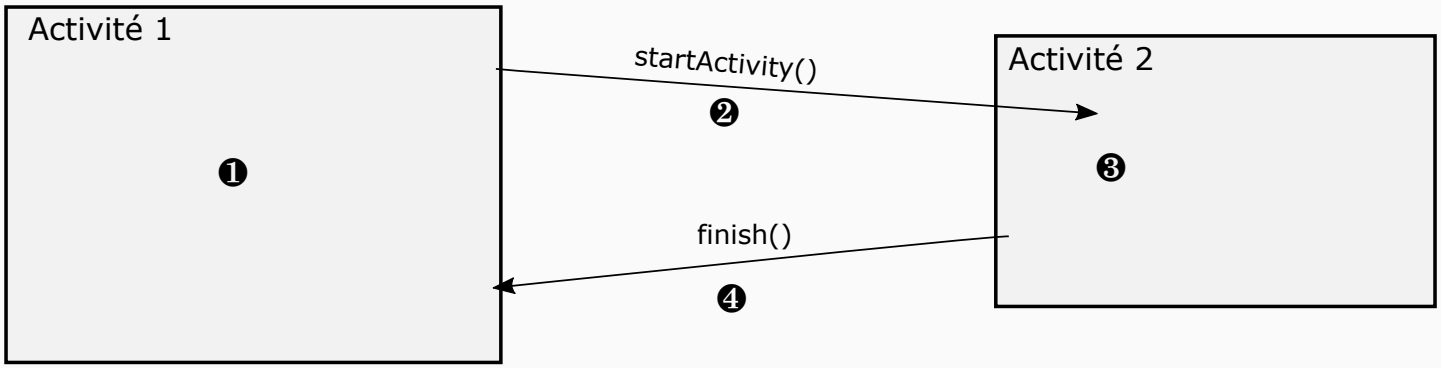
- constructeur permettant de désigner l'activité cible :

```
Intent(Context packageContext, Class<?> cls)
```

- exemple :

```
val context = LocalContext.current
val versAutre = Intent(context, SecondActivity::class.java)
startActivity(versAutre)
```

`Intent` implicite



implicite :

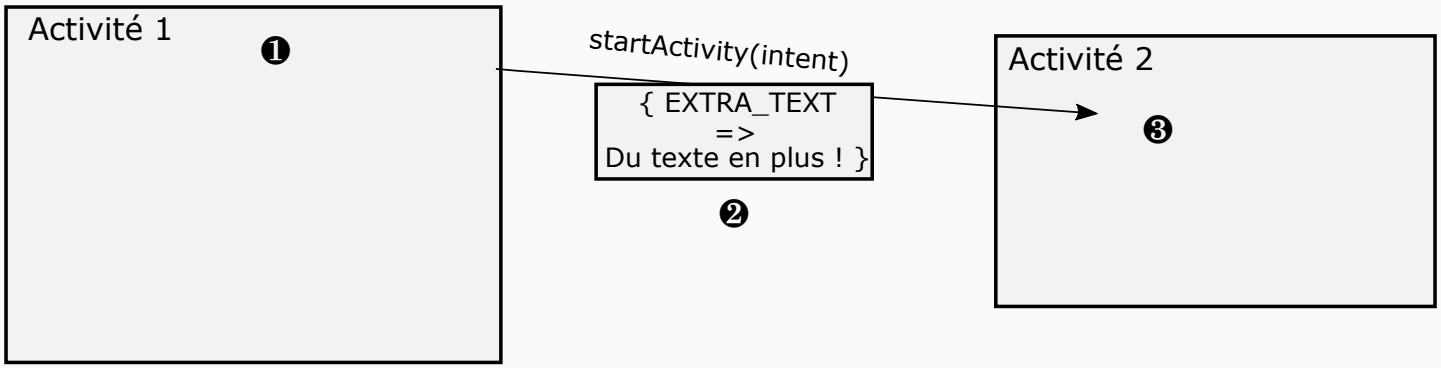
- constructeurs permettant de désigner action et/ou données :

```
Intent(String action)
Intent(String action, Uri uri)
```

- exemple :

```
val c = Calendar.getInstance()
val versAutre = Intent(AlarmClock.ACTION_SET_ALARM)
    .putExtra(AlarmClock.EXTRA_MESSAGE, "Demo")
    .putExtra(AlarmClock.EXTRA_HOUR, c[Calendar.HOUR_OF_DAY])
    .putExtra(AlarmClock.EXTRA_MINUTES, c[Calendar.MINUTE] + 1)
startActivity(versAutre)
```

`Intent` : avec données



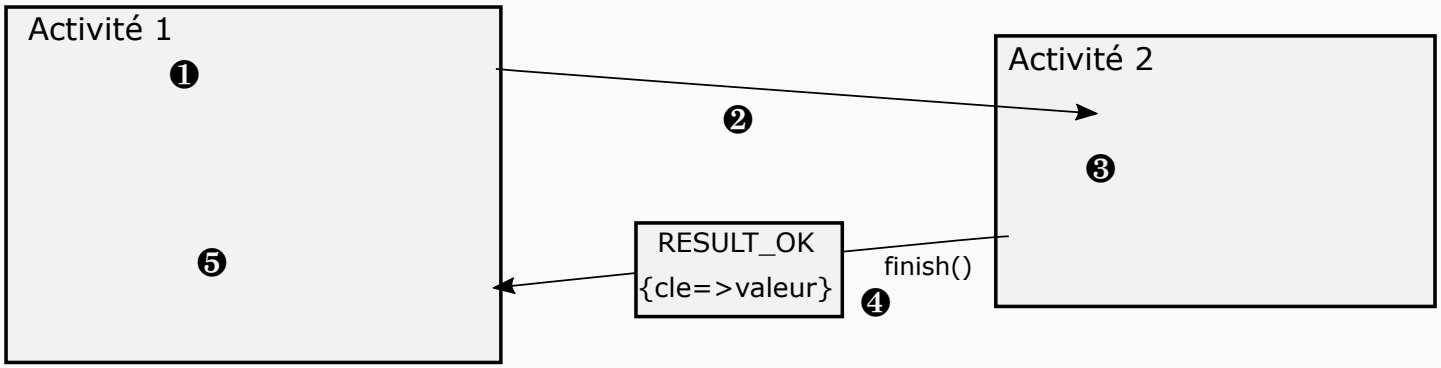
- dans l'Activité 1, on passe une map dans `Intent`
- utilisation de : `putExtra(clé,valeur)`
  clé : `String`, valeur : type multiples (primitifs, `String`, tableau, `Parcelable`, ...)

```
val intent = Intent(...)
intent.putExtra(Intent.EXTRA_TEXT, "Du texte en plus !")
startActivity(intent)
```

- dans l'Activité 2, on accède aux données par :

```
val valeur = intent.getStringExtra(Intent.EXTRA_TEXT)
// intent.getParcelableExtra(EXTRA_XXX, XXX::class.java)
```

`Intent` : avec réponse 1/3



- utilisation d'un `Contrat` qui est une sous-classe de `ActivityResultContract<I, O>`. Par ex : `GetContent` sous-classe de `ActivityResultContract<String, Uri?>`
- mise en oeuvre :
  1. dans l'Activité 1, enregistrement d'un contrat par `rememberLauncherForActivityResult<In,Out>(unContrat)`
  2. toujours dans l'enregistrement, définition du traitement du résultat par une lambda.

```
getContent = rememberLauncherForActivityResult(GetContent()) {
  uri: Uri? -> println(uri) }
```

  3. quand souhaité, lancement de l'Activité 2 : `launch(données)`

```
Button(onClick = { getContent.launch("image/*") })
```

`Intent` : avec réponse 2/3

- dans l'Activité 2, on utilise soit :
  - `setResult(int)` pour un code de retour (dont éventuellement `Activity.RESULT_CANCELED`, `RESULT_OK`)
  - `setResult(int, Intent)` si on veut transmettre un résultat en plus du code de retour.
  - puis `finish()`
- dans l'Activité 1, le retour se fait dans la lambda enregistrée

`Intent` : avec réponse 3/3

- Contrats :
  - Il existe un contrat générique `StartActivityForResult` : (n'importe quel) `Intent` en entrée, `ActivityResult` en sortie (attributs `resultCode ; Int` et `data : Intent?`) → calque le fonctionnement antérieur
  - Contrats existant : `CaptureVideo`, `PickContact`, `TakePicture`, `RequestPermission`, ...
  - il est possible de définir un contrat personnalisé :
    1. sous classer `ActivityResultContract<I,O>`
    2. redéfinir
       `fun createIntent(context: Context, input: I): Inten`
       comment produit-on un `Intent` à partir de notre entrée
    3. redéfinir
       `fun parseResult(resultCode: Int, intent: Intent?):`
       comment extrait-on notre sortie d'intent
  - `I` et `O` sont à remplacer par les types de votre choix : `String`, `Void?`, `Int?`, ...

`Intent` courants

- quelques combinaisons usuelles
- email : action `ACTION_SENDTO`, extra `Intent.EXTRA_EMAIL` (To), `Intent.EXTRA_SUBJECT`, `Intent.EXTRA_TEXT`
- map : action `ACTION_VIEW`, data `geo:0,0?q=mon+addresse`
- téléphone : action `ACTION_DIAL`, data `tel:+numéro`
- photo : action `ACTION_IMAGE_CAPTURE`
- contact : action `ACTION_PICK`
- ...

`http://developer.android.com/guide/components/intents-common.html`

Intent Filter

- permettent de déclarer les capacités de nos activités
- utilisés dans les résolutions des `Intent`s implicites
- déclarés dans le manifest
- critères :
  1. action

     `<action android:name="XXX"/>`

     `"android.intent.action.AAA"` = `Intent.ACTION_AAA`

     `"android.intent.action.[PICK|PASTE|ANSWER|...]"`

     `"android.media.action.IMAGE_CAPTURE"`
  2. catégorie

     `<category android:name="XXX"/>`

     `android.intent.category.[DEFAULT|BROWSABLE|TAB|ALTERNAT`

     `LAUNCHER|HOME|PREFERENCE|TEST]`
  3. données + type : certains aspects de l'URI de la donnée (scheme, host, port, path, etc.) et type MIME

     ```
     <data android:scheme="XXX" android:host="XXX"
           android:port="XXX" android:path="XXX"
           android:pathPattern="XXX"
           android:pathPrefix="strXXXing"
     ```

Intent Filter : exemples

1. cas usuel : activité déclarant une application "principale"

```
<intent-filter>
    <action android:name="android.intent.action.MAIN" />
    <category android:name="android.intent.category.LAUNCHER" />
</intent-filter>
```

2. activité sachant lire et éditer des images png

```
<intent-filter>
    <action android:name="android.intent.action.VIEW" />
    <action android:name="android.intent.action.EDIT" />
    <data android:mimeType="image/png" />
</intent-filter>
```

3. à votre avis ?

```
<intent-filter>
    <action android:name="android.intent.action.VIEW" />
    <data android:type="audio/*" />
    <data android:type="application/ogg" />
    <category android:name="android.intent.category.DEFAULT" />
</intent-filter>
```

UI : Lists

- pour afficher une collection d'éléments, on peut se contenter de `Column` et `Row`

```
@Composable
fun MessageList(messages: List<Message>) {
    Column {
        messages.forEach { message ->
            MessageRow(message)
        }
    }
}
```

- éventuellement en utilisant un modificateur `verticalScroll()` ou `horizontalScroll`
- en cas de grand nombre d'éléments → problème de performance
- solution : listes différées ou *lazy lists*

UI : `LazyColumn` & `LazyRow`

- équivalent du `RecyclerView` du système des Views : ne compose et ne dessine que les éléments visibles à l'écran.
- idéal pour les listes volumineuses ou de taille inconnue.
- pas de modificateur `XXXScroll`, le défilement est géré nativement.
- fonctions décrivant la mise en page : `item()`, `items(Int)` et `items(List)`

```
LazyColumn (
    verticalArrangement = Arrangement.spacedBy(4.dp),
    contentPadding = PaddingValues(horizontal = 16.dp, vertical =
)
{
    item {
      Text("En-tête de liste")
    }
    items(3) { index ->
        Text(text = "Item: $index")
    }
    items(itemsList) { item ->
        ItemCard(item)
```

UI : `rememberLazyListState`

- permet de remonter l'état pour contrôler ou observer le scroll.
- utile pour bouton "Retour en haut" ou retrouver la position de scroll
- `LazyListState` fournit également `firstVisibleItemIndex`

```
val listState = rememberLazyListState()

LazyColumn(state = listState) {
  /* ... */
}
```

- `firstVisibleItemIndex` permet de réagir à la position actuelle.

UI : Optimisation via `key`

- par défaut, Compose identifie un item par sa **position**.
- si la liste change (ajout/suppression au milieu), tout est recomposé.
- solution : fournir une clé unique et stable.

```
LazyColumn {
    items(
        items = personList,
        key = { person -> person.id } // Stable & Unique
    ) { person ->
        PersonRow(person)
    }
}
```

UI : Listes

- **Erreur classique :**

```
val list = remember { mutableStateOf(mutableListOf<T>()) }
```

- Le changement de contenu interne (`list.add()`) ne déclenche pas de recomposition !
- Il faut utiliser des types observables spécifiques :

```
// conversion d'une liste existante
val list = remember { initialList.toMutableStateList() }

// déclaration directe
val list = remember { mutableStateListOf<Item>() }
```

- Toute opération (`add`, `remove`, `clear`) sera alors détectée par Compose.

Coroutines : Principes

Les coroutines sont un système de gestion de tâches

- **exécution des tâches :**
  - de manière **concurrente** : utilisation de threads
  - en **parallèle** (sous certaines conditions)
  - **sans blocage**
- **avantages :**
  - gestion de l'asynchronisme de manière (presque) séquentielle
  - les coroutines sont coopératives

Coroutines : Scopes et Dispatchers

- **Scopes :** durée de vie
  - `viewModelScope` : pour la logique métier (ViewModel).
  - `lifecycleScope` : pour le code lié à l'Activity.
- **Dispatchers :** gestion du pool de threads d'exécution
  - `Dispatchers.Main` : UI (par défaut en Compose). Un seul thread.
  - `Dispatchers.IO` : Réseau, Base de données. Pool de threads nombreux car ils passent leur temps à attendre le réseau ou le disque.
  - `Dispatchers.Default` : Calculs lourds. Pool de threads limité au nombre de coeurs du processeur.
- Toujours déléguer les tâches lourdes au dispatcher IO ou Default.

Coroutines : `suspend`

- mot-clé **`suspend`** est le marqueur d'une coroutine
- peut interrompre son exécution sans bloquer le thread d'exécution
- fonctionnement :
  1. la fonction s'arrête à un point précis (ex : un appel réseau).
  2. le thread est libéré pour faire autre chose (ex : une écriture base de données)
  3. une fois le résultat disponible, la coroutine reprend où elle s'était arrêtée.
- une fonction **`suspend`** ne peut être appelée que depuis une autre fonction **`suspend`** ou depuis une coroutine.
- constructeur de coroutines : `launch`, `async` et `runBlocking` (NON)

Coroutines : `LauchedEffect`

- pour lancer une coroutine liée au cycle de vie d'un composant
- s'annule quand le composant quitte la composition

```
@Composable
fun SimpleCounter() {
    var text by remember { mutableStateOf("") }
    var isTooLong by remember { mutableStateOf(false) }
    val snackbarHostState = remember { SnackbarHostState() }

    isTooLong = text.length > 10

    LaunchedEffect(isTooLong) {
        if (isTooLong) {
            snackbarHostState.showSnackbar("Attention : Texte
        }
    }
```

Coroutines : `rememberCoroutineScope`

- utilisé pour lancer des coroutines depuis des **callbacks** (clics).
- le scope est lié au cycle de vie de l'appelant.

```
@Composable
fun Screen() {
    val scope = rememberCoroutineScope()
    val snackbarHostState = remember { SnackbarHostState() }

    Button(onClick = {
        // Obligatoire pour appeler des suspend fun au clic
        scope.launch {
            snackbarHostState.showSnackbar("Message envoyé !")
        }
    }) {
        Text("Afficher Snackbar")
    }
}
```

Réseau

- sources diverses :
  1. `java.net` : `Socket`, `URL` ($\longrightarrow$ IO) : laborieux pour HTTP
  2. `org.apache` : `HttpRequest`, `HttpResponse` : dépréciés
  3. `android.net` : `URI`, `java.net` : `HttpURLConnection`
  4. bibliothèques tierces :
     - *Volley*
       (`https://developer.android.com/training/volley/index.html`)
     - *Ion*
       (`https://github.com/koush/ion`)
     - *Retrofit*/*okHttp*
       (`https://square.github.io/retrofit`)
       (`https://square.github.io/okhttp`)
       (`ktor`)
     - ...
- nécessite des permissions :

```
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.ACCESS_NETWO
```

Json

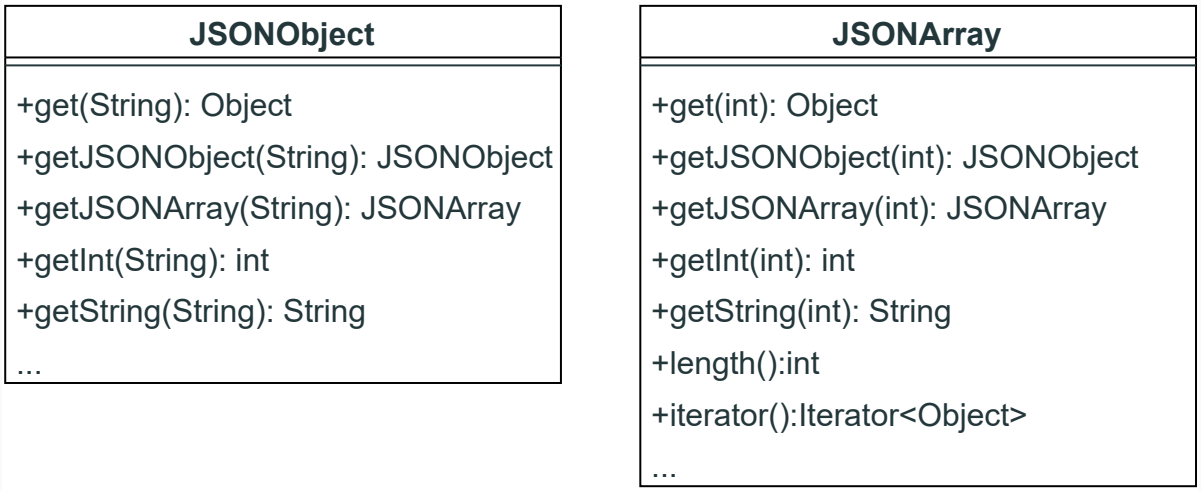
- données packagées en deux structures :
  1. couples clé/valeur
  2. listes ordonnées de valeur
- exemple :

```
{
 "myNumber": 123.456,
 "myString": "abcdef",
 "myBool": true,
 "myArray": [ "a", "b", "c", "d" ],
 "myObject": { "id": 7 },
 "myNull": null
}
```

https://www.json.org/json-fr.html

Json

org.json

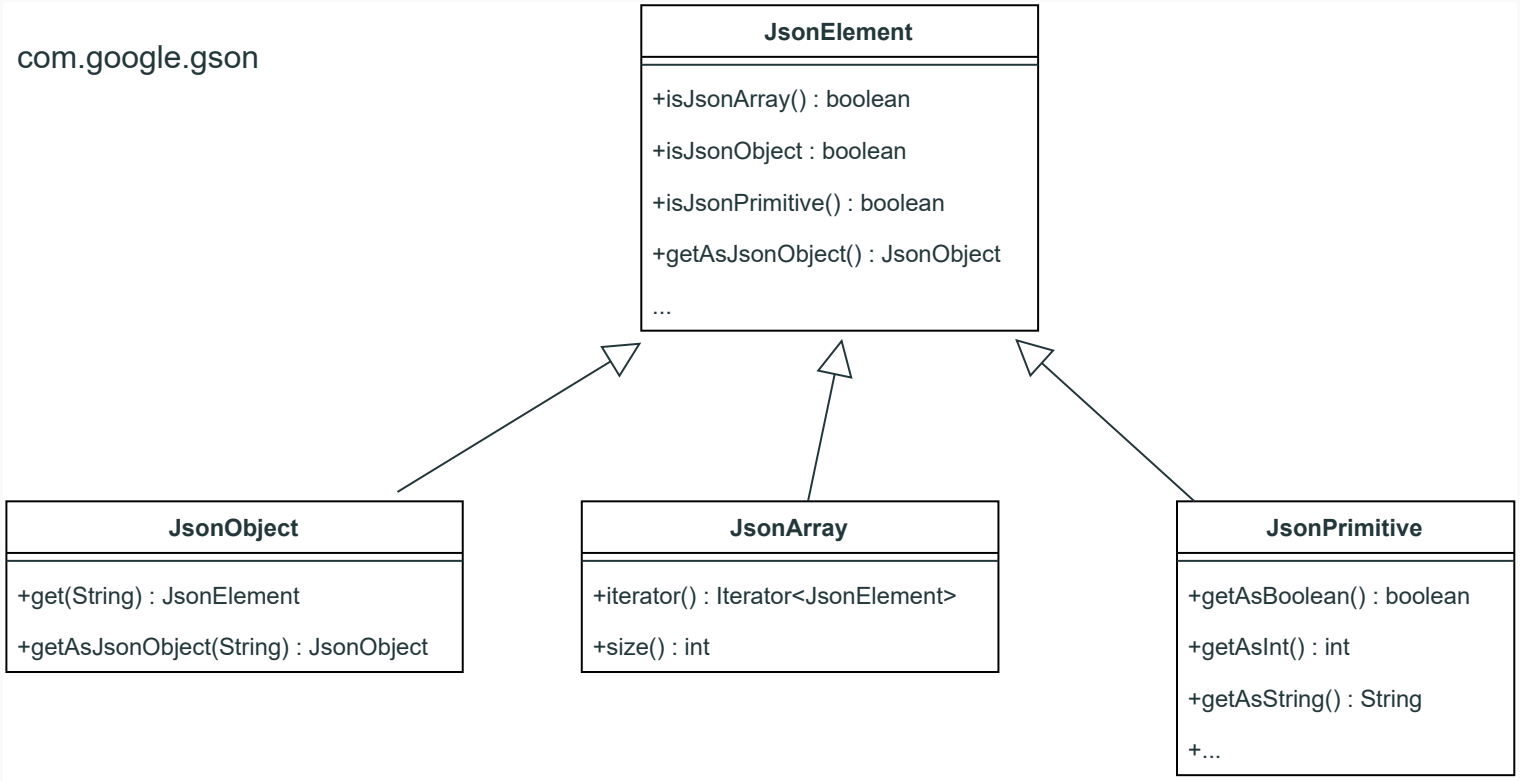


Json : exemple

```
https://github.com/stleary/JSON-java
```

```
// TODO
```

Json : API concurentes



Json String $\longleftrightarrow$ Pojo (*Plain old Java object*) / kotlin *data class*

- gson (`https://github.com/google/gson`),
- moshi (`https://github.com/square/moshi`),
- jackson (`https://github.com/FasterXML/jackson`),
- Koltinx-serialisation-json (`https://github.com/Kotlin/kotlinx.serialization`)

Persistance

Plusieurs solutions possibles :

1. fichier : privé à l'application ou partagé
2. map : association clés/valeurs improprement appelée *Shared preferences* $\longrightarrow$ *DataStore*
3. SGBD sqlite : bases privées à l'application. API très bas niveau ou via un ORM : **Room**, OrmLite, ...
4. autres bd : Realm

Fichier

- il est posible d'utiliser l'API java `File`
- Création

```
val file = File(context.filesDir, filename)
```

- puis on utilise le stream approprié :

```
val fos = FileOutputStream(file)
```

- ou directement `openFileOutput()` de `Context`
- exemple :

```
val filename = "myfile"
val fileContents = "Hello world!"
context.openFileOutput(filename, Context.MODE_PRIVATE).use {
        it.write(fileContents.toByteArray())
}
```

https://developer.android.com/training/data-storage/app-specific

Shared preferences

- stockage d'une map clé-valeur
- limité aux types simples et chaîne
- écriture :

```
SharedPreferences sharedPref =
 context.getSharedPreferences("DEMO",Context.MODE_PRIVATE);
SharedPreferences.Editor editor = sharedPref.edit();
editor.putInt("CLICK", ++numClick);
editor.commit();
```

- lecture :

```
SharedPreferences sharedPref =
context.getSharedPreferences("DEMO",Context.MODE_PRIVATE);
int val = sharedPref.getInt("CLICK",-1);
```

- fichier XML (/data/data/dev.mobile.cmsp/shared_prefs/DEMO.xml)

```
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
    <int name="CLICK" value="2" />
</map>
```

Room

- Entity : définition du schéma de la base
- DAO (*Database Access Object*) : définition des opérations sur la base
- Database

Entity

- classe = table
- instance = ligne de donnée
- attribut = nom de colonne
- utilisation d'annotations
- getter/setter pour accès

```
@Entity(tableName = "demo")
data class Demo(
    @PrimaryKey(autoGenerate = true)
    val id: Int = 0,
    val name: String,
    val quantity: Int
)
```

https://developer.android.com/training/data-storage/room/defining-data

DAO

- classe d'accès aux données
- annotation :
  - @Delete : l'implantation de la méthode supprime ses paramètres de la base
  - @Insert : l'implantation de la méthode insère ses paramètres dans la base
  - @Query : requête ordinaire

```
@Dao
interface DemoDao {
    @Query("SELECT * FROM demo")
    fun getAll(): Flow<List<Demo>>

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insert(demo: Demo)

    @Query("DELETE FROM demo")
    suspend fun delete()

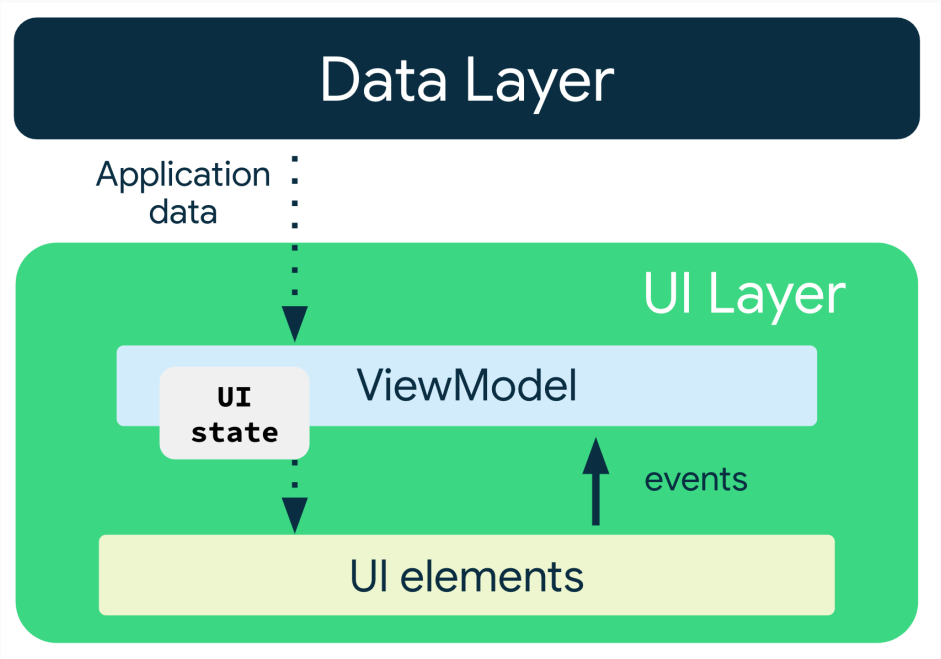
    @Query("SELECT * FROM demo")
    suspend fun getAllManually(): List<Demo>
```

Database

```
@Database(
    entities = [Demo::class],
    version = 1,
    exportSchema = false)

abstract class AppDatabase : RoomDatabase() {
    abstract fun demoDao(): DemoDao
    companion object {
        @Volatile
        private var Instance: AppDatabase? = null
        fun getDatabase(context: Context): AppDatabase {
            return Instance ?: synchronized(this) {
                Room.databaseBuilder(
                    context,
                    AppDatabase::class.java,
                    "demo_database"
                ).build().also { Instance = it }
            }
        }
    }
}
```

Architecture / UDF



- flux de données unidirectionnel (UDF) : l'état descend et les événements remontent
- le ViewModel contient et expose l'état à utiliser par l'UI
- l'état de l'UI = données d'application transformées par le ViewModel
- l'UI informe le ViewModel des événements utilisateur.
- le ViewModel gère les actions de l'utilisateur et met à jour l'état.
- L'état mis à jour est renvoyé à l'UI pour être affiché.

Conclusion ?

- ce que nous n'avons pas vu d'important :
  - `BroadcastReceiver`, `ContentProvider`, `Service`
  - capteurs : accéléromètre, gyroscope, électromagnétisme, orientation, ...
  - gestes et multitouch
  - Multimédia et graphisme
  - injection de dépendance par Hilt
  - navigation

Références

- https://developer.android.com/guide/index.html